

CODEALPHA • TASK 2 & TASK 3 | DATA ANALYTICS

# Titanic Passenger Survival Analytics

*A Comprehensive Data-Driven Investigation*

CodeAlpha Data Analytics Internship Portfolio (Task 2 & Task 3)

**Presented by Harsh**

Student ID: CA/DF1/264672



*891 passengers • 12 features • one question: who survived?*

# Gratitude & Internship Journey

## Thank you, CodeAlpha

Sincere thanks to CodeAlpha for the structured mentorship and a genuine project opportunity. The platform bridged theoretical data science with real-world exploratory data analysis (EDA), turning a classic dataset into a full analytical pipeline: from raw CSV to actionable, data-backed insight.

1

### Data wrangling

Missing-value audits, group-wise imputation, encoding

2

### Feature engineering

FamilySize, HasCabin, age-group segmentation

3

### Visual analytics

Seaborn / Matplotlib multi-plot figures

4

### Analytical storytelling

Turning statistics into a clear business narrative

# Mission, Scope & Core Objectives

891

passenger records

12

raw features

38.38%

baseline survival rate

## Analytical pipeline



### Core question

Which demographic and socio-economic factors (gender, passenger class, fare, family size, age) influenced who survived the sinking, and how strongly?

# Loading & Inspecting the Source Data

```
df = pd.read_csv('titanic.csv')
print(df.shape)    # (891, 12)
print(df.info())
```

## 891 × 12

rows × columns

## 3 dtypes

int64 · float64 · object

### Column blueprint

Numeric: PassengerId, Survived, Pclass, Age, SibSp, Parch, Fare. Object: Name, gender, Ticket, Cabin, Embarked.

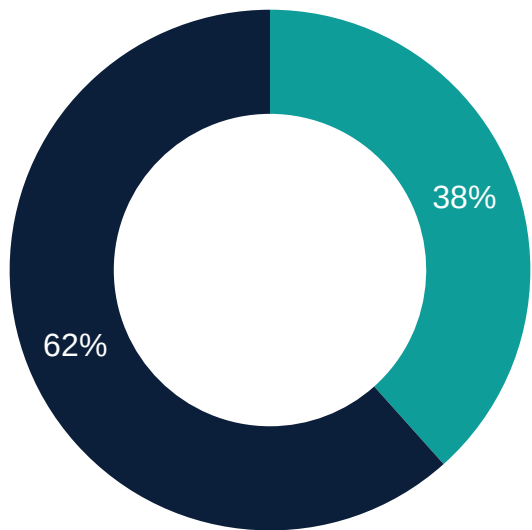
### df.head() preview

| ID | Survived | Pclass | gender | Age | Fare    |
|----|----------|--------|--------|-----|---------|
| 1  | 0        | 3      | male   | 22  | 7.25    |
| 2  | 1        | 1      | female | 38  | 71.2833 |
| 3  | 1        | 3      | female | 26  | 7.925   |
| 4  | 1        | 1      | female | 35  | 53.1    |
| 5  | 0        | 3      | male   | 35  | 8.05    |

### Why this matters

A first look at shape, types and sample rows sets expectations before any cleaning: Age has NaN values already visible.

# Understanding the Target: Survival Rate



■ Survived (342) ■ Did not survive (549)

**38.38%**

overall survival

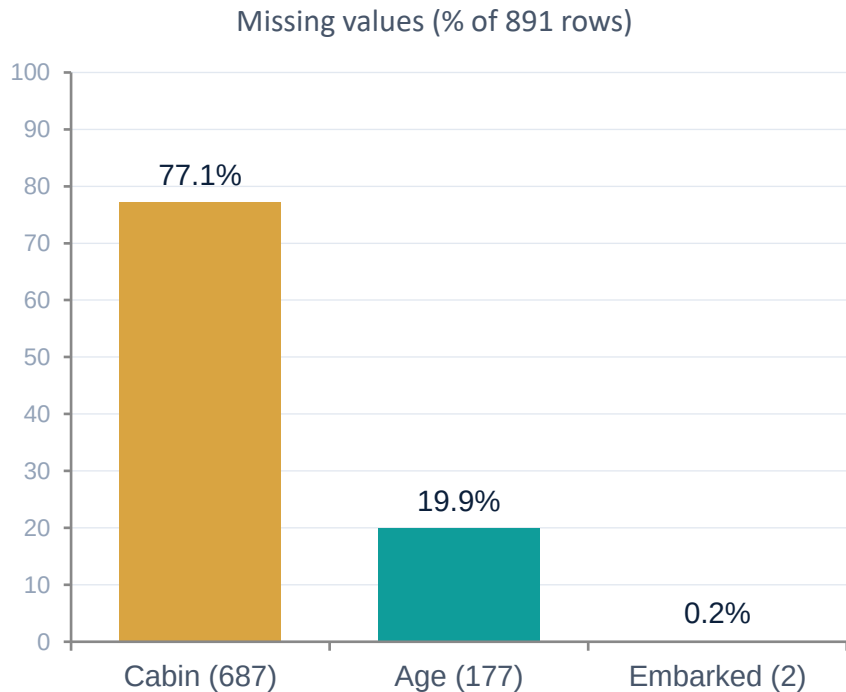
**342 / 549**

survivors / casualties

## Class imbalance

About 6 in 10 passengers died, a moderate imbalance. Accuracy alone would be misleading for future models: always predicting 'did not survive' scores 61.6%, so precision, recall and F1 must be reported.

# Identifying Data Gaps: Age, Cabin, Embarked



## Cabin: ~77% missing

Too sparse to impute. Convert to a binary HasCabin indicator instead.

## Age: 177 missing

Meaningful predictor, so impute with stratified medians (next slide).

## Embarked: 2 missing

Negligible: fill with the modal port.

# Solving Missing Age Data Rigorously

## Why not a global median?

One median (26.0) for everyone flattens variance and erases real structure: a 1st-class man is not the same age as a 3rd-class woman.

```
df['Age'] = df['Age'].fillna(  
    df.groupby(['Pclass', 'gender'])['Age']  
      .transform('median')  
)  
# vectorised, no loops
```

## Median age by Pclass × gender

| Class | Female | Male |
|-------|--------|------|
| 1     | 35.0   | 40.0 |
| 2     | 28.0   | 30.0 |
| 3     | 21.5   | 25.0 |

## Result

All 177 gaps filled with group-appropriate values while preserving the age distribution differences between classes.

# Data Type Optimization & Categorical Encoding

Object → numeric mappings

| Column           | Before        | After               |
|------------------|---------------|---------------------|
| gender           | male / female | 0 / 1               |
| Embarked         | S / C / Q     | one-hot flags       |
| Survived, Pclass | int64         | category (optional) |

```
df['Sex'] = df['gender'].map(  
    {'male': 0, 'female': 1})  
df = pd.get_dummies(  
    df, columns=['Embarked'])
```

## Outlier handling

Fare is heavily right-skewed. Flag outliers with the  $1.5 \times$  IQR rule and review them before deciding to cap or log-transform, rather than deleting valid first-class fares.

## Why encode?

Statistical and ML algorithms need numeric input. Binary and one-hot encodings also make correlation analysis possible on the same feature set, and lower memory usage.

# Engineering FamilySize & HasCabin

```
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1  
df['HasCabin']   = df['Cabin'].notnull().astype(int)
```

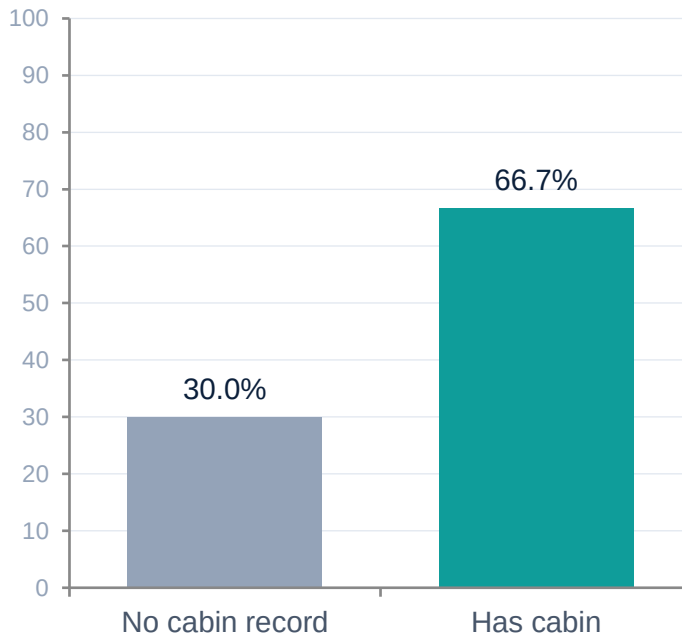
## FamilySize

SibSp + Parch + the passenger (+1). Captures the group a person had to evacuate with.

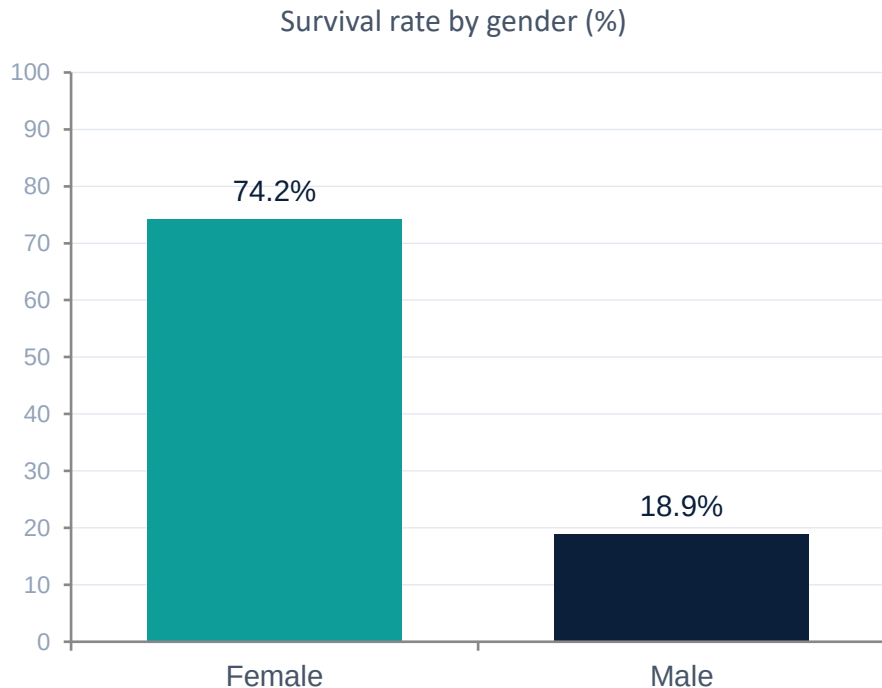
## HasCabin

Binary socioeconomic proxy: a recorded cabin usually meant a higher-deck, higher-class ticket.

Survival rate by HasCabin (%)



# Gender Disparity in Evacuation Protocols



**74.2%**

female survival

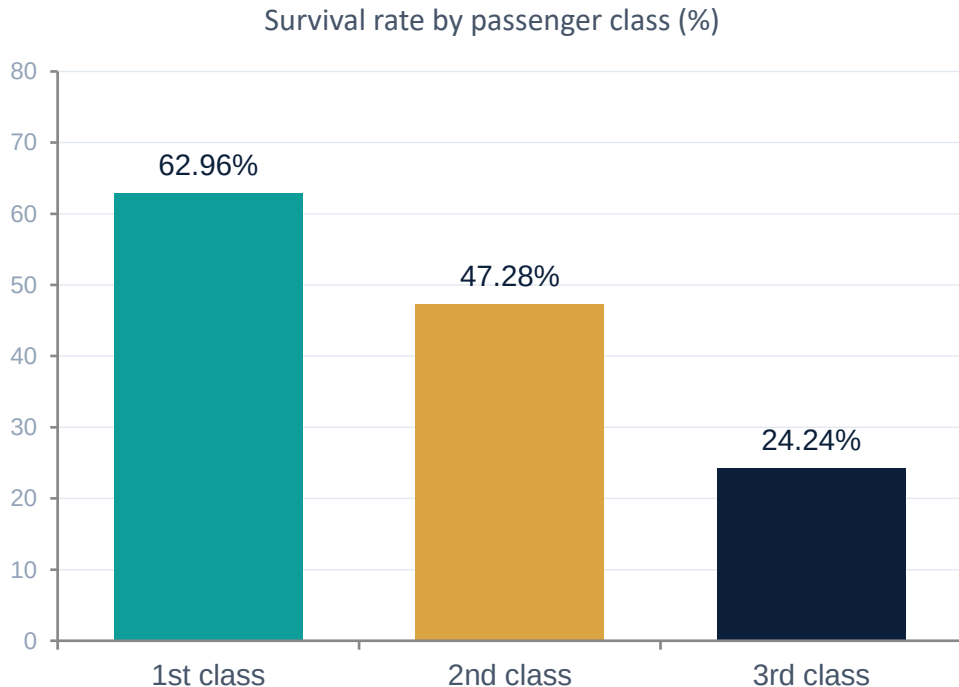
**18.9%**

male survival

## The 'women and children first' effect

A 55-point gap, the single strongest split in the data (correlation with Survived: +0.54). The historical evacuation mandate is visible directly in the numbers.

## The Class Gradient Factor (Pclass)



**63% vs 24%**

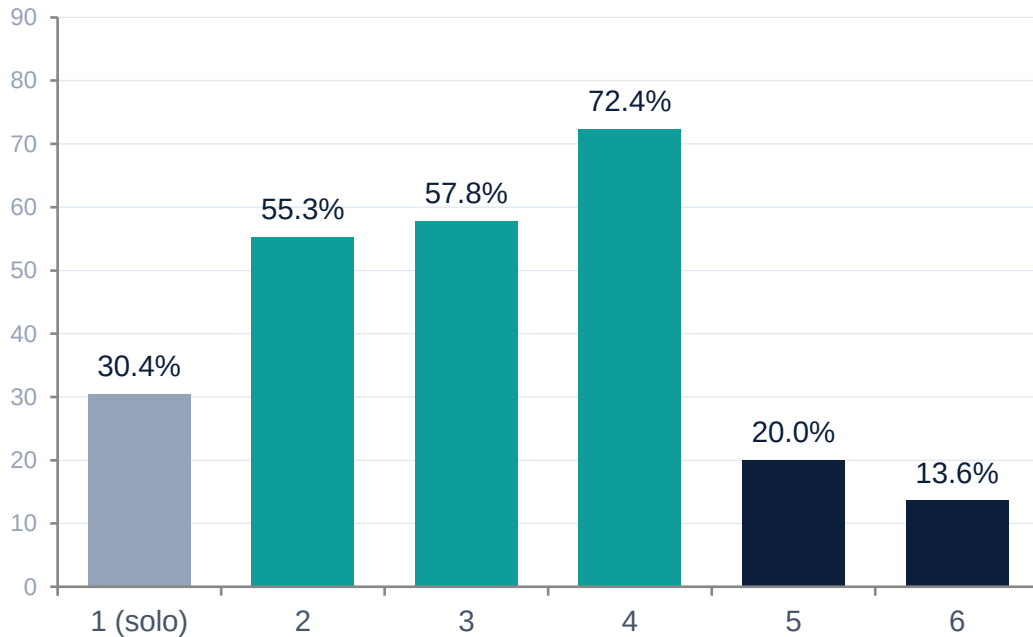
1st vs 3rd class survival

### Structural priority

Higher-deck cabins sat closer to the lifeboats. Survival falls monotonically with each class step, a 2.6× advantage for first class.

# Evacuation Efficiency vs. Family Bottlenecks

Survival rate by FamilySize (%)



## Solo: ~30%

No one to coordinate with or be prioritised alongside.

## 2–4: 55–72% sweet spot

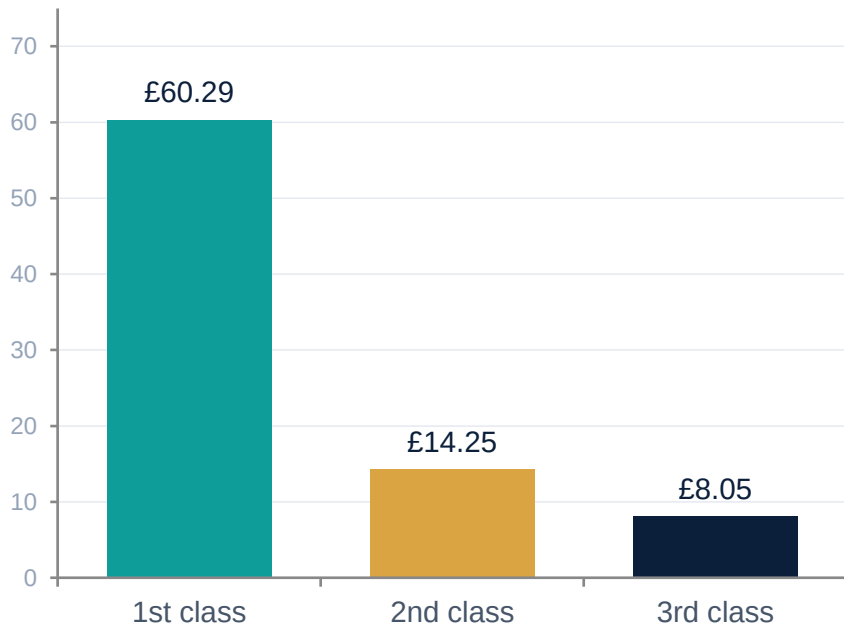
Small groups moved together and were kept together.

## 5+: gridlock

Large families lost time: 20% at 5, 13.6% at 6, ≤33% beyond.

# Economic Status & Fare Distribution Analysis

Median ticket fare by class (£)



**£60.29**

1st-class median

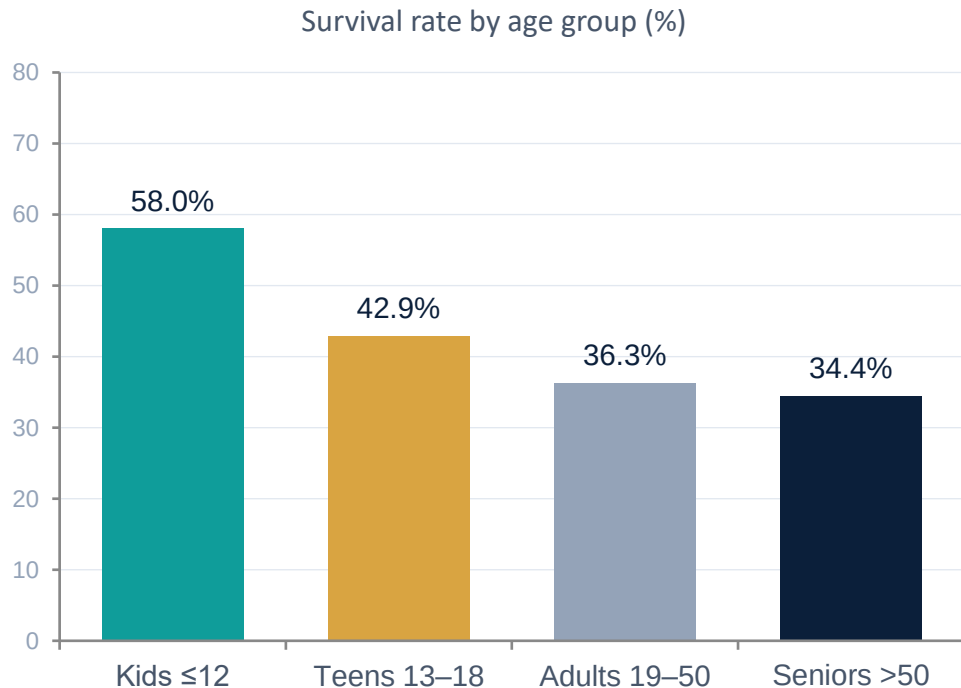
**£8.05**

3rd-class median

## Fare as a proxy for position

Fare is right-skewed with a long tail of expensive suites. It correlates +0.26 with survival and -0.55 with Pclass: higher fares meant better cabin positions, faster lifeboat access and higher odds. Boxplots by class (Seaborn) confirm the separation.

# Categorizing Life Stages: Kids to Seniors



## 100%

of 17 Class-2 children survived

### Anomaly worth noting

Kids overall survived at 58%, but only 41.7% in 3rd class (n=48). Class shaped even the 'children first' rule. Group sizes: 69 kids, 70 teens, 688 adults, 64 seniors.

## Cross-Tabulating Class and Gender

| Survival rate | Female | Male  |
|---------------|--------|-------|
| 1st class     | 96.8%  | 36.9% |
| 2nd class     | 92.1%  | 15.7% |
| 3rd class     | 50.0%  | 13.5% |

*Cells shaded by survival rate (darker = higher)*

### Gender beat class

A 3rd-class woman (50.0%) out-survived a 1st-class man (36.9%). Class 1 and 2 women survived at 92–97%, while 2nd and 3rd-class men sat below 16%. Gender protocol superseded privilege in the critical tiers, though class still set the floor for women.

# Feature Correlation Matrix Insights

|          | Survived | Pclass | Sex   | Age   | Fare  | Family |
|----------|----------|--------|-------|-------|-------|--------|
| Survived | 1.00     | -0.34  | 0.54  | -0.06 | 0.26  | 0.02   |
| Pclass   | -0.34    | 1.00   | -0.13 | -0.41 | -0.55 | 0.07   |
| Sex      | 0.54     | -0.13  | 1.00  | -0.10 | 0.18  | 0.20   |
| Age      | -0.06    | -0.41  | -0.10 | 1.00  | 0.12  | -0.26  |
| Fare     | 0.26     | -0.55  | 0.18  | 0.12  | 1.00  | 0.22   |
| Family   | 0.02     | 0.07   | 0.20  | -0.26 | 0.22  | 1.00   |

Teal = positive, gold = negative (Sex: 1 = female)

## Positive with Survived

Sex +0.54 (strongest), Fare +0.26.

## Negative with Survived

Pclass -0.34, Age -0.06 (weak).

## Structure

Fare vs Pclass -0.55: money and class move together.

# What Harsh Learned: Technical & Analytical Takeaways

1

## Advanced Pandas vectorization

groupby + transform replaces slow row loops.

2

## Multi-plot figure architecture

Seaborn / Matplotlib subplots that tell one story.

3

## Avoiding silent data errors

Checking shapes, NaNs and dtypes after every transform.

4

## Production-ready code

Clear English comments, reusable structure, reproducible runs.

# Clean Code Standards in Python (.ipynb Blueprint)

```
def load_data(path):  
    """Read CSV and validate shape."""  
    df = pd.read_csv(path)  
    assert df.shape == (891, 12)  
    return df  
  
def clean(df):  
    """Impute Age by Pclass x gender."""  
    df['Age'] = df['Age'].fillna(  
        df.groupby(['Pclass', 'gender'])['Age']  
        .transform('median'))  
    return df  
  
def engineer(df):  
    df['FamilySize'] = df.SibSp + df.Parch + 1  
    df['HasCabin'] = df.Cabin.notnull().astype(int)  
    return df
```

## Clear English comments

Every step says what and why.

## Modular functions

load → clean → engineer → plot.

## Exception-safe pipeline

Asserts and try/except guard against missing files and unexpected schemas.

# Final Analytical Verdict

*Socio-economic status (Pclass) and gender (Sex) were the dominant determinants of survival, far outweighing individual age or family size.*

**74.2% vs 18.9%**

female vs male

**63.0% vs 24.2%**

1st vs 3rd class

**+0.54 / -0.34**

Sex / Pclass vs Survived



Family size mattered mainly at the extremes: solo travelers and families of 5+ fared worst.

# Next Steps: From EDA to Predictive Modeling

Convert this exploratory script into a full ML classification pipeline that predicts survival on unseen test data.

Logistic Regression

Decision Tree

Random Forest

*Train/test split → cross-validation → precision, recall, F1*

**Thank you | Harsh**

CA/DF1/264672 · CodeAlpha Data Analytics Internship

